

ECO-LOOP BUILDING AGENTS

AI that thinks before your building wastes energy.

An open-source LLM closes the control loop on a live EnergyPlus simulation — set-points injected into the running instance, savings proven against an identical baseline.

PROBLEM STATEMENT

ID — «fill in from portal»
Title — Eco-Loop Building Agents: autonomous closed-loop building control (Honeywell Hackathon · Question 1)
Theme — «fill in from portal»
PS Category — Software

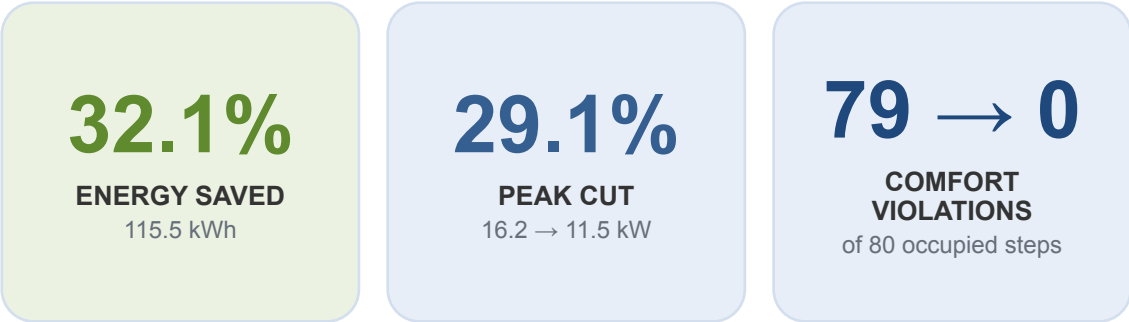
TEAM

Student Name (registered on portal) — Pranav Kumar
Student ID — «fill in from portal»

IN THE REPOSITORY

Closed-loop source · baseline and runtime-generated .idf models · savings dashboard · docs/ARCHITECTURE.md · 392 passing tests

MEASURED ON REAL ENERGYPLUS — NOT PROJECTED



✓ COMFORT MAINTAINED · ASHRAE-55 PMV BAND HELD ALL DAY

EnergyPlus 26.1 · summer week · 192 timesteps of 15 min

Agent run and fixed-schedule baseline execute the identical code path, weather and occupancy — a controlled experiment, reproducible from two CLI commands.

EnergyPlus 26.1

Llama 3.3 70B via Groq

MCP server

FastAPI + SSE

React + Recharts

392 tests passing

ECO-LOOP BUILDING AGENTS

Buildings waste energy because their controls cannot think. Ours reasons — every fifteen minutes.

THE PROBLEM

- ✗ **Buildings burn ~40% of global energy** and remain a primary driver of carbon emissions.
- ✗ **Building management systems run fixed schedules** blind to weather, occupancy and grid carbon intensity.
- ✗ **Control is reactive, not anticipatory** comfort is corrected only after occupants have complained.

79 of 80

occupied timesteps outside the comfort band — measured on the fixed-schedule baseline in EnergyPlus.

OUR SOLUTION



AI SUPERVISOR

Llama 3.3 70B reads live telemetry and sets the control policy.



ENERGYPLUS DIGITAL TWIN

Set-points injected into the running instance — no restart, no file rewrite.



REACTIVE SAFETY GUARD

Every action clamped in code to hard comfort and equipment limits.



MCP TOOL LAYER

The same six tools serve the agent and any external MCP client.



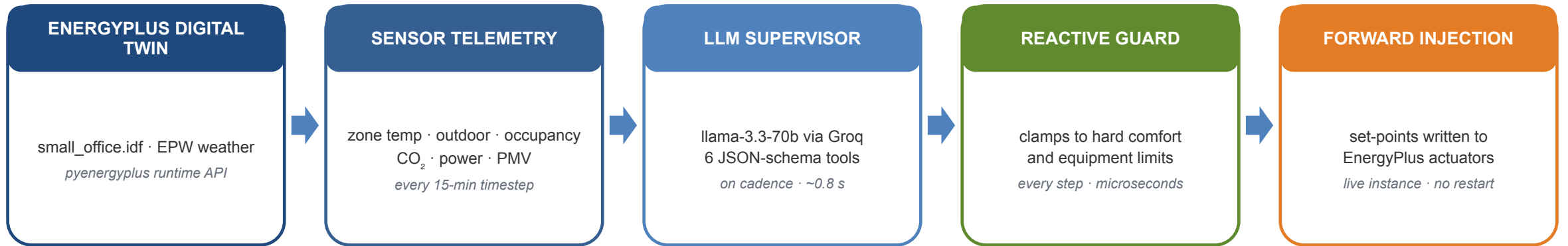
LIVE SAVINGS DASHBOARD

Eight panels streaming telemetry, reasoning and kWh saved.

WHAT MAKES IT DIFFERENT · baseline and agent run the identical code path, weather and occupancy — so the saving is a controlled experiment, not a demo

TECHNICAL APPROACH

One closed loop: sense → reason → clamp → actuate, inside the running simulation.



CLOSED LOOP — the next timestep's telemetry already reflects the action just written. 35,040 steps (a full simulated year) completed end to end.

EVIDENCE STORE

SQLite: runs · timesteps · decisions. 39,690 telemetry rows and 5,021 decisions persisted, with rationale, latency and guard overrides.

API LAYER

FastAPI — 14 REST endpoints plus a Server-Sent Events stream and per-run CSV export. The dashboard is a pure reader.

MCP SERVER

The agent's own ToolRegistry served over stdio, so an external MCP client drives the identical six tools.

Python 3.12

EnergyPlus 26.1

pyenergyplus

Groq · Llama 3.3

MCP

FastAPI

React + Vite

Recharts

392 tests

FEASIBILITY AND VIABILITY

It already runs — and it is engineered to fail safely.



RUNS TODAY

Real EnergyPlus 26.1 and real Groq calls, measured at 465–2,449 ms per supervisory decision. Two CLI commands reproduce the headline number.



TESTED

392 automated tests pass — 161 backend (pytest) and 231 frontend (vitest), including the EnergyPlus thread bridge and forward injection.



EVIDENCED

39 recorded runs · 39,690 telemetry rows · 5,021 decisions, each with its rationale and latency. Every run exports to CSV.

RISK → OUR ANSWER

Model latency on a 35,040-step year



Two-tier control. The guard runs every step in microseconds; the LLM runs on a cadence and only chooses the policy. The loop never waits on a model.

A hallucinated or unsafe set-point



Safety lives in code, not in a prompt. Every action is clamped to hard comfort and equipment limits, and each override is recorded as guard_clamped.

Hosted API rate-limits, times out, or fails



Bounded retries → circuit breaker → baseline fallback through the same guard. One 192-step run completed with 44 of 48 supervisory calls degraded and zero crashes.

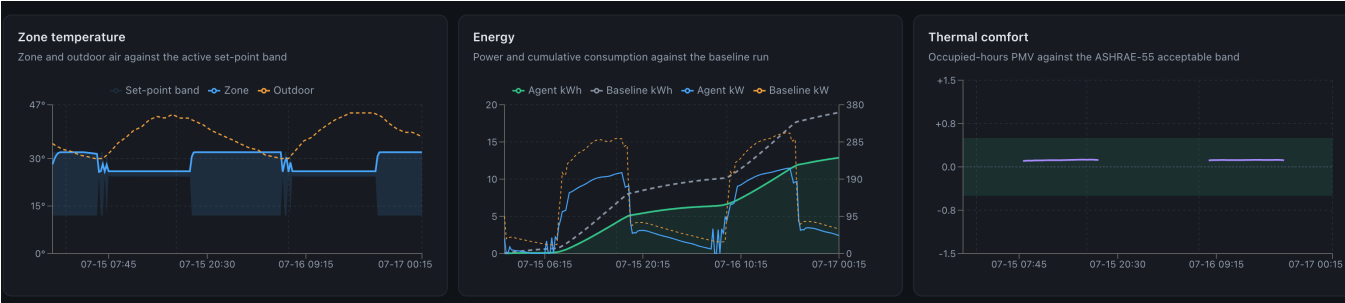
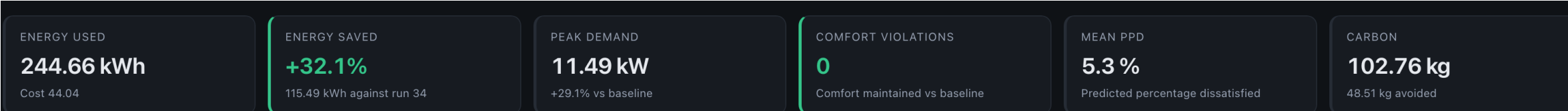
Logs and telemetry exceed any context window



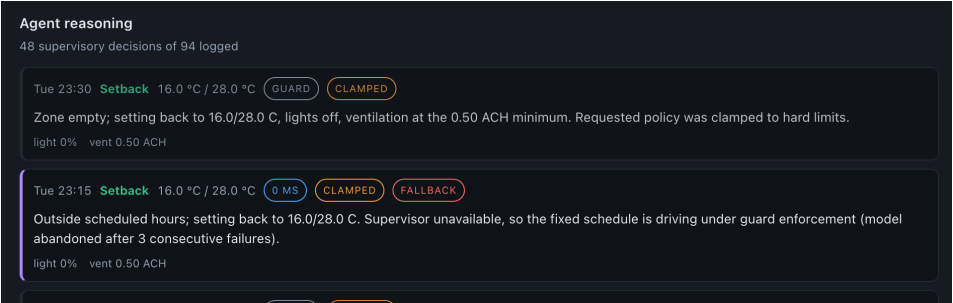
logsummary.py compacts before prompting: severity filtering, warning de-duplication and statistical windowing of long traces.

ARTIFACTS

The live dashboard on the real run: 32.1% less energy, zero comfort violations.



Live telemetry — zone temperature against the set-point band · agent vs baseline energy · PMV inside the ASHRAE-55 band



Every decision explains itself — and the guard's overrides are on the record



REPRODUCE THE HEADLINE NUMBER

```
python cli.py --scenario summer_week --controller baseline --simulator energyplus --days 2
python cli.py --scenario summer_week --controller rule --simulator energyplus --days 2 --compare
<id>
```

AGENT DECISION — VERBATIM FROM THE RUN LOG

“The zone is unoccupied and will remain so for the next 4 steps, so we can set back the temperature and turn off the lighting to save energy.”

set_control_policy · setback · 19.0 / 20.5 °C · 849 ms · llama-3.3-70b-versatile

Artifacts: backend/ (24 modules) · frontend/ (9 components) · small_office.idf + runtime-generated variants · docs/ARCHITECTURE.md · 392 tests

RESEARCH AND REFERENCES

WHY THIS SOLUTION WINS — six claims, each backed by something in the repository.

REAL AI

An open-weight Llama 3.3 70B reasons over live telemetry and emits a policy through native tool-calling.

REAL SIMULATION

EnergyPlus 26.1 driven through its runtime API — actuators written mid-run, not a file regenerated between runs.

EXPLAINABLE

Every decision persists its rationale, tool call, latency and any guard override. The dashboard shows them live.

SAFE

A deterministic guard clamps every action to hard comfort and equipment limits, so a hallucinated set-point cannot reach the building.

SCALABLE

The simulator sits behind a Protocol and the model is named in .env — neither vendor is locked in; the MCP server opens the tools to any client.

PRODUCTION-READY

392 passing tests, a typed API, an evidence store and CSV export — a controlled experiment anyone can re-run.

RESEARCH AND REFERENCES

STANDARDS

- Fanger PMV / PPD thermal comfort — ISO 7730 Annex D and ANSI/ASHRAE Standard 55; implemented in app/comfort.py.
- EnergyPlus Weather (EPW) hourly format — parsed in app/sim/weather.py.

SIMULATION ENGINE

- EnergyPlus 26.1 and its runtime Python API (pyenergyplus), NREL — github.com/NREL/EnergyPlus/releases
- Baseline model derived from the distribution example MovableExtInsulationSimple.idf.

MODEL AND PROTOCOL

- Groq inference API serving open-weight Llama 3.3 70B (llama-3.3-70b-versatile) — console.groq.com
- Model Context Protocol — official Python SDK (mcp ≥ 1.0), wrapping the agent's own ToolRegistry.

Full design rationale, sequence and data-flow diagrams, database schema and design decisions D1–D11: [docs/ARCHITECTURE.md](https://docs.groq.com/ARCHITECTURE.md)